

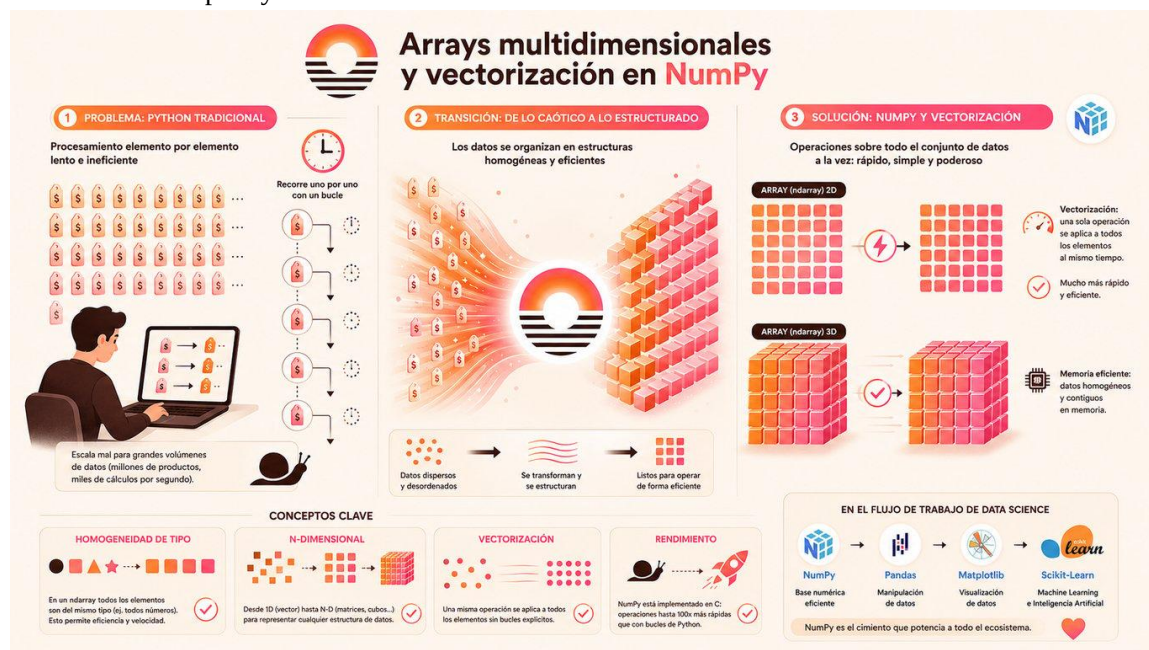
Arrays multidimensionales y vectorización en NumPy

Arrays multidimensionales y vectorización en NumPy

Imagina que trabajas en una tienda que vende 500 productos diferentes. Cada día, recibes una lista con los precios de costo y necesitas calcular el precio de venta final sumándoles un impuesto del 21%.

Si usaras lo que hemos aprendido de Python "puro" hasta ahora, escribirías un bucle `for` que recorra la lista, tome cada precio, le sume el impuesto y lo guarde en una nueva lista. Para 500 productos, no hay problema. Pero, ¿y si fueras una cadena global con 10 millones de productos y realizaras miles de cálculos por segundo? Tu código se volvería extremadamente lento.

Aquí es donde entra **NumPy**. En esta unidad, descubriremos por qué NumPy es el cimiento de casi toda la Ciencia de Datos en Python y cómo nos permite tratar miles de datos como si fueran uno solo, de forma increíblemente rápida y sencilla.



2. ¿Por qué existe NumPy en el flujo de trabajo de datos?

Python es un lenguaje fantástico por su legibilidad, pero no fue diseñado originalmente para el cálculo matemático de alto rendimiento. Las listas de Python son flexibles (pueden guardar un número, un texto y un booleano al mismo tiempo), pero esa flexibilidad tiene un costo: **lentitud y alto consumo de memoria**. **NumPy** (Numerical Python) nació para solucionar esto. Es una biblioteca que permite realizar operaciones matemáticas sobre grandes colecciones de datos de forma eficiente.

El problema que resuelve:

1. **Rendimiento**: Las operaciones en NumPy están escritas en lenguajes de bajo nivel (como C), lo que las hace hasta 100 veces más rápidas que los bucles tradicionales de Python.
2. **Sintaxis Matemática**: Nos permite escribir fórmulas matemáticas directamente sobre los conjuntos de datos, sin necesidad de "abrir" el conjunto elemento por elemento.

En el flujo de trabajo de un Científico de Datos, NumPy es la herramienta que está "debajo del capó" de casi todo lo demás. **Pandas** (para tablas), **Matplotlib** (para gráficos) y **Scikit-Learn** (para Inteligencia Artificial) dependen de NumPy para funcionar.

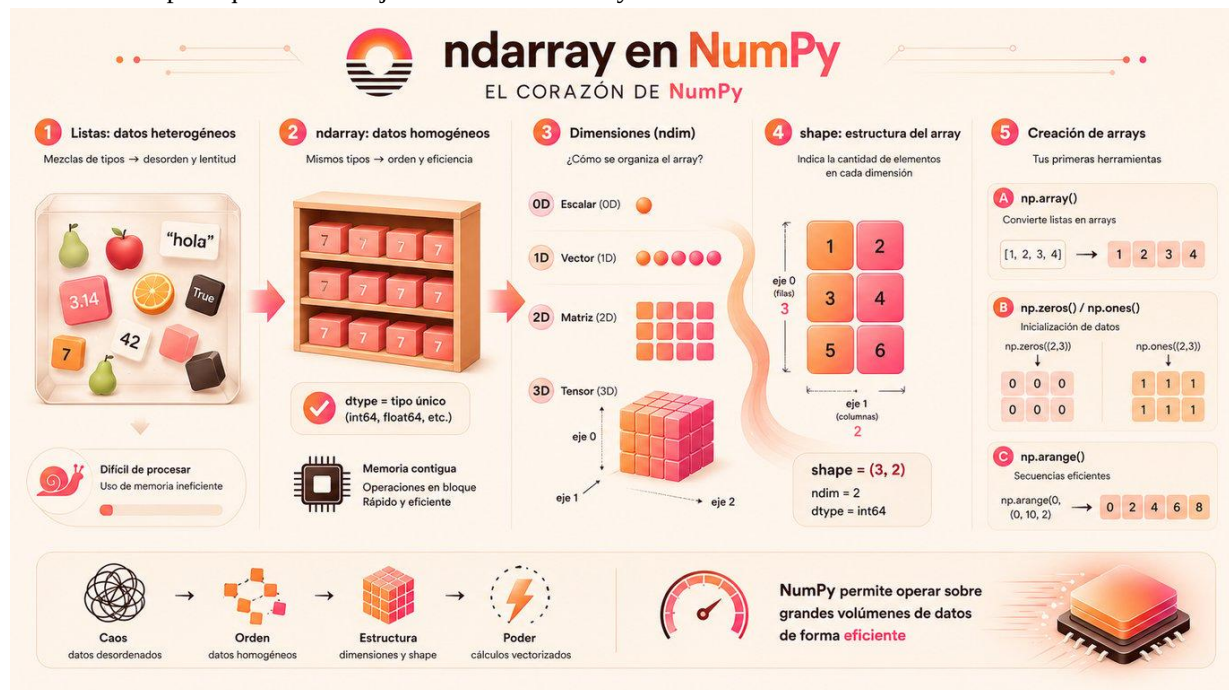
3. ¿Qué es un ndarray? El corazón de NumPy

El objeto principal de NumPy es el ndarray (n-dimensional array). Aunque a simple vista se parece a una lista de Python, tiene una regla de oro: la **homogeneidad de tipo**.

Homogeneidad de tipo

A diferencia de una lista de Python, donde puedes mezclar peras con manzanas, en un array de NumPy **todos los elementos deben ser del mismo tipo** (por ejemplo, todos números enteros o todos flotantes).

¿Por qué es esto importante? Si NumPy sabe que "todo lo que hay en esta caja es un número entero de 64 bits", puede reservar un bloque exacto de memoria y realizar cálculos en bloque. Es como tener una estantería diseñada específicamente para cajas de zapatos del mismo tamaño: puedes apilarlas y moverlas mucho más rápido que si cada caja tuviera un tamaño y forma distinta.



4. Dimensiones, ejes y forma (ndim y shape)

Para trabajar con datos, necesitamos entender cómo están organizados espacialmente. NumPy usa tres conceptos clave para describir un array:

- 1. Dimensiones (ndim):** Es el número de "ejes" que tiene el array.
 - **0D (Escalar):** Un solo número (ej. 5).
 - **1D (Vector):** Una fila de números (como una lista simple).
 - **2D (Matriz):** Una tabla con filas y columnas.
 - **3D (Tensor):** Un "cubo" de datos (como una hoja de cálculo con varias pestañas, o una imagen a color que tiene Alto, Ancho y Capas de color).
- 2. Forma (shape):** Es una tupla que nos dice cuántos elementos hay en cada dimensión.
 - Un array de (5,) tiene 5 elementos en una sola dimensión.
 - Un array de (3, 2) tiene 3 filas y 2 columnas.
- 3. Tipo de dato (dtype):** Nos dice si los datos son `int64` (enteros), `float64` (decimales), etc.

Ejemplo visual:

- **Un punto** es 0D.
- **Una línea** de puntos es 1D.
- **Un cuadrado** (filas y columnas) es 2D.

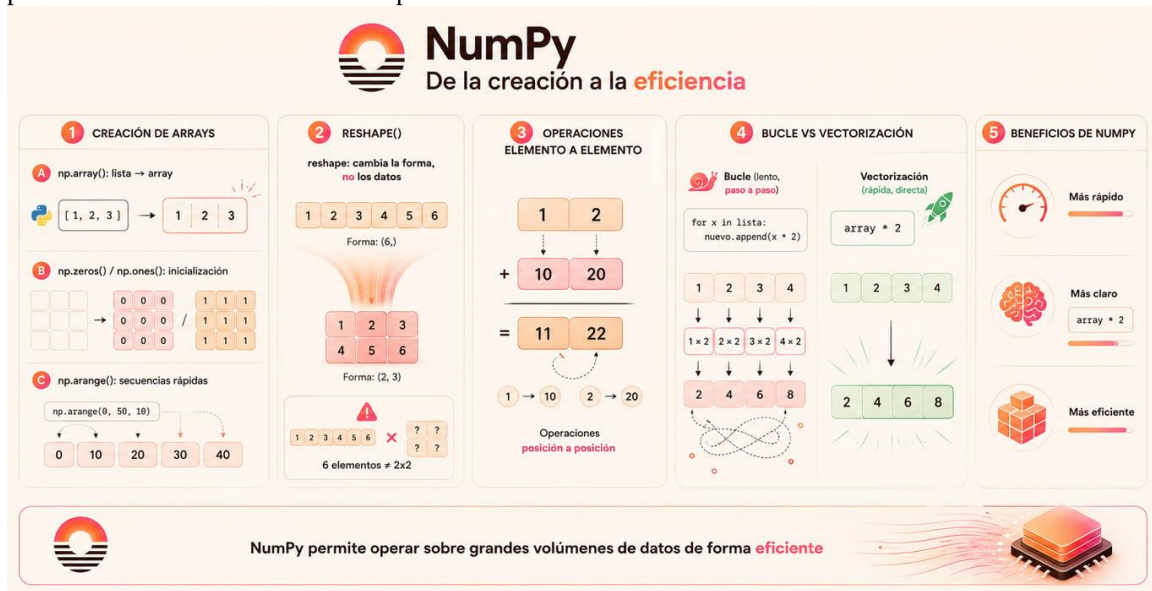
- Un cubo (pilas de cuadrados) es 3D.

5. Creación de arrays: tus primeras herramientas

Existen varias formas de "dar vida" a un array en NumPy:

- **np.array()**: Convierte una lista de Python existente en un array.
python

```
import numpy as np
mi_lista = [1, 2, 3]
mi_array = np.array(mi_lista)
```
- **np.zeros()** y **np.ones()**: Crea arrays llenos de ceros o unos. Es muy útil para "preparar" el espacio donde luego guardaremos resultados de cálculos.
- **np.arange(inicio, fin, paso)**: Es como el `range()` de Python, pero devuelve un array. Ideal para crear secuencias numéricas rápidas.



6. Cambio de forma y operaciones elemento a elemento

reshape: Reorganizando los datos

A veces recibimos datos en una sola línea (1D) pero sabemos que representan una tabla (2D).

Con **reshape**, podemos cambiar la estructura sin alterar los datos. **Importante:** El número total de elementos debe coincidir. No puedes convertir 6 elementos en una tabla de 2x2 (porque sobrarían 2).

Operaciones elemento a elemento

En NumPy, si sumas dos arrays, la operación ocurre automáticamente "posición a posición". Si

tienes `array_A = [1, 2]` y `array_B = [10, 20]`, hacer `array_A + array_B` te dará `[11, 22]`. **¡Sin bucles for!**

7. Vectorización frente a bucles Python

Este es el concepto más importante de esta unidad. La **vectorización** es la capacidad de aplicar una operación a todo un array simultáneamente.

El enfoque tradicional (Bucle):

Para duplicar los valores de una lista, tendrías que:

1. Crear una lista vacía.

2. Iterar por cada elemento.
3. Multiplicar por 2.
4. Agregar a la nueva lista.

El enfoque NumPy (Vectorización):

Simplemente haces: `mi_array * 2`.

¿Por qué es mejor?

1. **Claridad:** El código es más corto y se parece más a una fórmula matemática real.
 2. **Velocidad:** NumPy le dice al procesador de tu computadora: "Toma todo este bloque de números y multiplícalos todos de una vez", en lugar de pedirle que haga miles de tareas pequeñas una tras otra.
-

8. Errores comunes y mejores prácticas

- **Error: Mezclar tipos.** Si creas un array con `[1, 2, "3"]`, NumPy convertirá todo a texto (`string`) para mantener la homogeneidad, y ya no podrás hacer cálculos matemáticos. **Siempre verifica tu `dtype`.**
 - **Error: Confundir `shape` con `len()`.** En una matriz de 4x5, `len()` solo te dirá 4 (el número de filas). Para saber el tamaño total o la estructura completa, usa siempre `.shape`.
 - **Práctica recomendada:** Evita los bucles `for` siempre que trabajes con datos numéricos. Si sientes la tentación de escribir un `for` para procesar un array, detente y busca si existe una función de NumPy que lo haga de forma vectorizada.
-

9. Síntesis de la unidad

En esta unidad hemos sentado las bases del cómputo científico:

- Aprendimos que **NumPy** es esencial por su velocidad y eficiencia en memoria.
- Entendimos que el **`ndarray`** es una estructura robusta y **homogénea** (un solo tipo de dato).
- Vimos que los datos tienen "forma" (`shape`) y dimensiones (`ndim`), lo cual es vital para organizar información compleja.
- Descubrimos la **Vectorización**: nuestra superpotencia para eliminar bucles lentos y escribir código elegante y rápido.

En la próxima unidad, profundizaremos en cómo estas matrices interactúan entre sí cuando tienen tamaños diferentes mediante una técnica llamada *Broadcasting*.

Para practicar (No entregable):

Propósito del ejercicio

Practicar la creación de arrays y experimentar de primera mano la diferencia entre el pensamiento algorítmico tradicional (bucles) y el pensamiento orientado a datos (vectorización).

Instrucciones

1. **Preparación:** Crea una lista de Python llamada `precios_lista` que contenga 10 números decimales (pueden ser precios ficticios).
2. **Conversión y Exploración:**
 - Convierte esa lista en un array de NumPy llamado `precios_array`.
 - Imprime en pantalla la **forma** (`shape`), el **número de dimensiones** (`ndim`) y el **tipo de dato** (`dtype`) del array.
3. **Transformación:**
 - Usa el método `reshape` para cambiar tu array de 10 elementos a una matriz de 5 filas y 2 columnas (5, 2).
4. **Operaciones Vectorizadas:**

- Crea un nuevo array llamado `precios_con_impuesto` multiplicando el array original por 1.21 (simulando un IVA del 21%).
 - Suma 5 unidades a cada elemento del array resultante.
 - Calcula el cuadrado de cada elemento (elevado a la potencia 2).
5. **El Desafío de Comparación:**
- Implementa un bucle `for` de Python que recorra la lista original `precios_lista`, le sume el 21% de impuesto a cada elemento y guarde el resultado en una nueva lista.
 - Escribe un breve comentario en tu código comparando la cantidad de líneas y la "limpieza" del código del bucle frente a la operación vectorizada de NumPy.

Errores comunes a evitar

- **Incompatibilidad en reshape:** Asegúrate de que el producto de las nuevas dimensiones (ej. $5 * 2$) sea exactamente igual al número de elementos originales (10).
- **Olvidar el import:** Recuerda que para usar NumPy debes primero importar la librería con `import numpy as np`.

✦ **Entregable de este módulo:** Checkpoint — Estructura inicial del dataset del proyecto (evaluable, suma al proyecto final). Su consigna y rúbrica están en la unidad de pre-entrega de este mismo módulo.

Broadcasting y operaciones matemáticas sobre matrices

Broadcasting y Operaciones Matemáticas sobre Matrices en NumPy

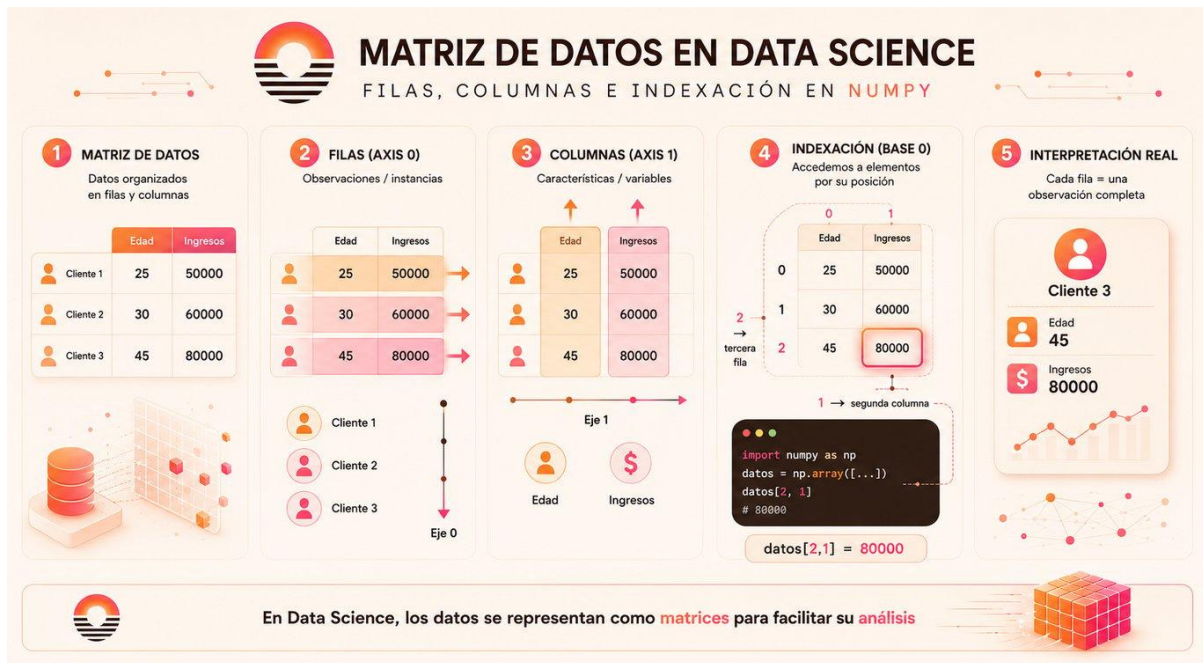
Objetivos de la Unidad

- Comprender la estructura de una Matriz de Datos en proyectos reales.
- Dominar el concepto de **Broadcasting** para operar con arrays de distintas dimensiones.
- Diferenciar las operaciones elemento a elemento de la **Multiplicación Matricial**.
- Aplicar estas herramientas a casos de uso reales en la Ciencia de Datos.

Introducción

En la unidad anterior aprendimos qué es un `ndarray` y cómo realizar operaciones elemento a elemento sencillas. Sin embargo, en el mundo real de la Ciencia de Datos, raramente trabajamos con arreglos del mismo tamaño exacto.

Imagina que tienes una tabla con las alturas de 1,000 personas agrupadas por género y quieres restarle el promedio específico a cada grupo. ¿Tendrías que crear un arreglo de 1,000 promedios idénticos o usar un bucle `for` que haga lento tu código? **No**. Para eso existe el verdadero "superpoder" de NumPy: el **Broadcasting**.



1. El Concepto de Matriz de Datos

En Data Science, la información casi siempre se organiza de forma tabular, formando lo que llamamos una **Matriz de Datos**.

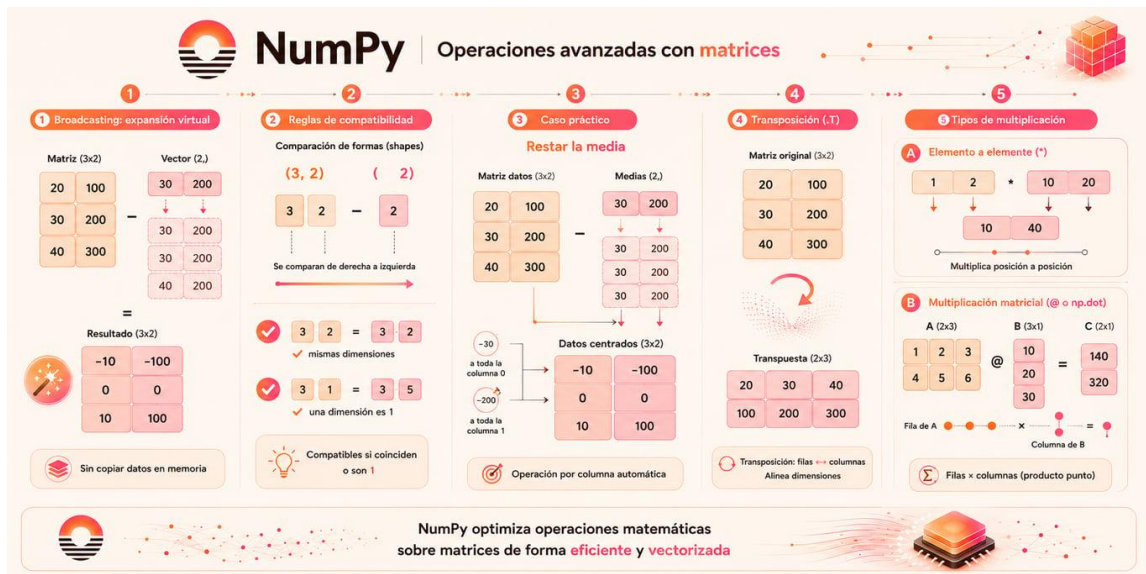
- **Filas (eje 0):** Representan cada *observación* o *instancia*. Puede ser un cliente, un sensor, una transacción o un día.
- **Columnas (eje 1):** Representan las *características* o *variables* de esa observación (edad, temperatura, precio, etc.).

Recordatorio de indexación: En Python empezamos a contar desde 0. Por lo tanto, un elemento en la posición (2, 1) de una matriz NumPy se refiere a la **tercera fila** y **segunda columna**.

```
import numpy as np
# Matriz de 3 observaciones (filas) y 2 características (columnas)
datos = np.array([ [25, 50000], # Cliente 1: 25 años, $50,000 ingresos
                  [30, 60000], # Cliente 2: 30 años, $60,000 ingresos
                  [45, 80000] ]) # Cliente 3: 45 años, $80,000 ingresos
print(datos[2, 1]) # Salida: 80000 (Tercera fila, segunda columna)
```

2. Broadcasting: El Superpoder de NumPy

El **Broadcasting** es un conjunto de reglas que permite a NumPy realizar operaciones aritméticas entre arrays de diferentes formas (*shapes*), siempre que sean compatibles. Esto nos evita escribir código repetitivo y optimiza el uso de memoria, ya que las operaciones se ejecutan en C (el lenguaje base de NumPy) de forma ultrarrápida.



¿Cómo funciona?

Cuando intentas operar con dos arrays de distintos tamaños, NumPy intenta "estirar" o "replicar" virtualmente el más pequeño para que coincida con el más grande, sin copiar realmente los datos en la memoria.

Reglas de Compatibilidad

NumPy compara las dimensiones de los arrays de **derecha a izquierda**. Dos dimensiones son compatibles cuando:

1. **Son exactamente iguales.**
2. **Una de ellas es 1.**

Caso Práctico: Restar la media

Supongamos que tienes una matriz de datos de 3x2 y un vector con las medias de cada columna de tamaño 2. NumPy "estirará" el vector de medias hacia abajo para que cubra las 3 filas, permitiendo restar la media correspondiente a cada elemento sin usar un solo bucle.

```
# Matriz 3x2 (3 clientes, 2 variables)
matriz_datos = np.array([[20, 100], [30, 200], [40, 300]])

# Vector de medias (tamaño 2)
medias = np.array([30, 200])

# Aplicando Broadcasting: restamos el vector a toda la
matriz_datos_centrados = matriz_datos - medias

print(datos_centrados) # Salida: # [[-10 -100] # [ 0 0] # [ 10 100]]
```

3. Transposición y Multiplicación Matricial

Es crucial en Data Science no confundir la multiplicación elemento a elemento con la Multiplicación Matricial propia del álgebra lineal.

Multiplicación elemento a elemento (*)

El operador `*` multiplica la posición (0,0) de la primera matriz por la (0,0) de la segunda matriz, la (0,1) con la (0,1), etc. Las dimensiones deben ser compatibles según las reglas de broadcasting.

Transposición (.T)

Gira la matriz, convirtiendo las filas en columnas. Si tienes una matriz de 3x2, su transpuesta será de 2x3. Esto es una herramienta fundamental para alinear dimensiones antes de hacer cálculos pesados.

Multiplicación Matricial (`np.dot` o `@`)

Sigue la regla clásica del álgebra lineal (fila por columna). Para multiplicar la Matriz A por la Matriz B, **el número de columnas de A debe ser igual al número de filas de B**.

Python

```
A = np.array([[1, 2], [3, 4], [5, 6]]) # Shape: 3x2
B = np.array([[10, 20], [30, 40]]) # Shape: 2x2
# Multiplicación Matricial (3x2 @ 2x2 = 3x2) resultado = A @ B
print(resultado)
```

4. Importancia en la Industria (Casos de Uso Reales)

¿Para qué usamos esto en el día a día como Data Scientists?

1. **Escalado de datos:** Ajustar una columna entera de precios por la inflación mensual usando un solo escalar (ej. `precios * 1.05`).
2. **Normalización (Z-score):** Para que los algoritmos de Machine Learning funcionen bien, solemos restar el promedio y dividir por la desviación estándar a conjuntos enteros de datos de millones de filas. Todo esto se hace en una sola línea gracias al broadcasting puro: `(datos - medias) / desviaciones`.
3. **Modelos Lineales y Redes Neuronales:** Las predicciones de una Regresión Lineal o de una capa de Red Neuronal se calculan internamente mediante una multiplicación matricial entre los datos de entrada y los pesos (`weights`) del modelo.

5. Errores Comunes de la Trinchera

- **✗ ValueError: operands could not be broadcast together with shapes...**: Ocurre cuando intentas sumar o multiplicar con `*` arrays que no cumplen la regla de compatibilidad (ej. intentar sumar un array de 3 elementos con uno de 4).
- **✗ ValueError: frames are not aligned/shapes not aligned:** Ocurre al usar `@` o `np.dot` cuando las dimensiones internas no coinciden. Por ejemplo, intentar multiplicar una matriz de 3×2 por otra de 3×2 sin transponer la segunda. La solución suele ser usar `.T` (ej. `A @ B.T`).
- **✗ Confundir `*` con `@`:** Recuerda, `*` es para ajustes elemento a elemento. `@` es para combinar características con pesos en álgebra lineal.

Mini-Desafío Práctico

Tu turno: Tienes las calificaciones de 4 alumnos en 3 exámenes (Matriz 4×3). El profesor ha decidido dar puntos extra diferentes para cada examen: +2 al primero, +1 al segundo y +3 al tercero (Vector de 3 elementos).

Tarea: Usa NumPy y el concepto de Broadcasting para sumar los puntos extra a todos los alumnos en una sola línea de código. ¡Anímate a probarlo en tu notebook!

Álgebra lineal con NumPy

Fundamentos de Álgebra Lineal Computacional con NumPy

1. Operaciones Matriciales y Vectorizadas

En Ciencia de Datos, los algoritmos rara vez procesan datos de forma aislada. NumPy utiliza el objeto `ndarray` (bloques de memoria contiguos) para ejecutar operaciones en paralelo mediante **vectorización**, eliminando la necesidad de bucles iterativos (`for loops`) en Python y delegando el cálculo a rutinas optimizadas en C y Fortran (BLAS/LAPACK).

A. Producto Punto (Dot Product)

El producto punto es una operación algebraica que toma dos vectores de la misma longitud y devuelve un único valor escalar. Multiplica los elementos correspondientes de cada vector y luego suma los resultados. Si $v = [v_1, v_2, \dots, v_n]$ y $w = [w_1, w_2, \dots, w_n]$, matemáticamente se define como:

$$v \cdot w = \sum_{i=1}^n v_i w_i$$

```
import numpy as np
# Definición de vectores (1D Arrays)v1 = np.array([1, 2, 3])v2 =
np.array([4, 5, 6])
# Cálculo del producto punto: (1*4) + (2*5) + (3*6) = 4 + 10 + 18 =
32dot_product = np.dot(v1, v2)print(f"Producto punto: {dot_product}")
```

B. Multiplicación Matricial

A diferencia de la multiplicación elemento por elemento ($A * B$), la multiplicación matricial combina filas y columnas. Para multiplicar una matriz A por una matriz B , el número de columnas de A **debe ser igual** al número de filas de B . El elemento en la posición (i, j) del resultado es el producto punto de la fila i de A y la columna j de B .

En NumPy moderno, se prefiere el operador infijo `@` sobre `np.dot()` para legibilidad del código.

Python

```
# Definición de matrices cuadradas (2D Arrays)A = np.array([[1, 2],
[3, 4]])
B = np.array([[5, 6],
[7, 8]])
# Multiplicación matricial estándarC = A @ B # Resultado: [[19, 22],
[43, 50]]
```

2. Transposición de Matrices

La transposición es una operación unaria que reorienta una matriz transformando sus filas en columnas y viceversa. Si una matriz original A tiene dimensiones $m \times n$, su transpuesta A^T tendrá dimensiones $n \times m$. Matemáticamente, el elemento a_{ij} se convierte en a_{ji} .

En NumPy, esta operación no copia los datos en memoria; simplemente cambia la forma en que el objeto *lee* el bloque de datos (cambia los *strides*), lo que la hace extremadamente eficiente.

Python

```
# Matriz original (2x2)A = np.array([[1, 2],
[3, 4]])
# TranspuestaA_T = A.T # Resultado: [[1, 3], [2, 4]]
```

3. Resolución de Sistemas de Ecuaciones Lineales

Muchos problemas de Ciencia de Datos se reducen a resolver un sistema de ecuaciones lineales del tipo:
 $Ax = b$

Donde:

- A es la matriz de coeficientes conocidos (dimensión $n \times n$).
- b es el vector de términos independientes (dimensión $n \times 1$).
- x es el vector de variables incógnitas que deseamos hallar.

El submódulo `np.linalg.solve`

Para encontrar x , matemáticamente podríamos pensar en calcular la inversa de la matriz ($x = A^{-1}b$). Sin embargo, **computacionalmente calcular la matriz inversa es ineficiente y propenso a errores de redondeo numérico** (inestabilidad). NumPy utiliza la descomposición LU (Factorización de matrices), que es mucho más rápida y estadísticamente estable.

Python

```
# Sistema: # 3x + 1y = 9# 1x + 2y = 8
A = np.array([[3, 1], [1, 2]])b = np.array([9, 8])
# Resolución óptima del sistemamax = np.linalg.solve(A, b) # Resultado:
[2., 3.] -> x = 2, y = 3
```

⚠ **Restricción Teórica:** Para que un sistema tenga una solución única mediante este método, la matriz A debe ser **cuadrada** (mismo número de ecuaciones que de variables) y **no singular** (su determinante debe ser distinto de cero, lo que significa que sus filas/columnas son linealmente independientes).

4. Diagnóstico y Propiedades Matriciales (`np.linalg`)

El módulo `np.linalg` provee las herramientas algebraicas fundamentales para inspeccionar las propiedades de nuestras estructuras de datos antes de someterlas a algoritmos de Machine Learning:

Función	Propiedad Matemática	Utilidad en Data Science
<code>np.linalg.det(A)</code>	Determinante: Valor escalar que evalúa si la matriz tiene inversa (si $\det = 0$, es singular).	Diagnóstico de colinealidad en variables.
<code>np.linalg.norm(v)</code>	Norma: Longitud o magnitud geométrica de un vector (por defecto, norma Euclídea o L_2).	Cálculo de distancias (ej. similitud coseno, algoritmos KNN).
<code>np.linalg.eig(A)</code>	Valores y Vectores Propios (Eigenvalues/Eigenvectors): Direcciones espaciales que no cambian con una transformación lineal, solo se escalan.	Reducción de dimensionalidad (PCA) y análisis de componentes principales.

Python

```
# Ejemplo de diagnóstico de propiedadesA = np.array([[4, 2],
[1, 3]])
# 1. Determinantedet_A = np.linalg.det(A)
# 2. Norma de un vectorvector = np.array([3, 4])norma =
np.linalg.norm(vector) # Resultado: 5.0 (Raíz de 3² + 4²)
# 3. Componentes principales (Eigendecomposition)valores_propios,
vectores_propios = np.linalg.eig(A)
```

5. Aplicaciones Reales en Ciencia de Datos

¿Por qué el Álgebra Lineal con NumPy es el motor de la disciplina?

- **Procesamiento de Features (Transformaciones Lineales):** Cuando normalizas datos, aplicas escala o rotas un conjunto de datos (como en visión por computadora al transformar imágenes), estás aplicando multiplicaciones matriciales sobre las coordenadas de los datos.
- **Regresión Lineal Analítica:** El ajuste de una regresión lineal por Mínimos Cuadrados Ordinarios (OLS) se calcula mediante la ecuación normal: $\beta = (X^T X)^{-1} X^T y$. Toda esta arquitectura requiere transpuestas, multiplicaciones y resolución de sistemas lineales que NumPy ejecuta internamente en milisegundos.
- **Representación de Datos de Alta Dimensión:** En flujos de trabajo con procesamiento de lenguaje natural (NLP) o sistemas de recomendación, los textos y usuarios se transforman en vectores de embeddings. El cálculo de similitud entre ellos se realiza vectorizando operaciones de producto punto y normas matriciales.

NumPy y Pandas: La Relación Entre el Cálculo y la Estructura de Datos

La Alianza del Dato: Entendiendo la Sinergia entre NumPy y Pandas

Imagina que tienes que organizar la información de una maratón internacional. Por un lado, tienes miles de tiempos de llegada (puros números) sobre los que necesitas calcular promedios y desviaciones en

milisegundos de forma instantánea. Por el otro, tienes una lista de corredores con nombres, edades, países de origen, tallas de camiseta y si pagaron o no la inscripción.

Si intentas usar listas básicas de Python para esto, te darás cuenta rápidamente de que son lentas para los cálculos matemáticos pesados y sumamente desordenadas para manejar tablas con diferentes tipos de datos. Aquí es donde entran **NumPy** y **Pandas**, las dos bibliotecas que transformaron a Python de un lenguaje de programación general en la herramienta reina de la Ciencia de Datos.

En esta unidad, no solo veremos qué son, sino cómo se complementan: **NumPy aporta el músculo matemático y Pandas aporta la inteligencia organizativa.**

1. El Motor de Cálculo: NumPy

NumPy (Numerical Python) es la biblioteca fundamental para la computación científica en Python. Si Python fuera un automóvil, NumPy sería un motor de carreras diseñado para procesar números a una velocidad que las listas normales no pueden alcanzar.

¿Por qué existe NumPy?

El problema con las listas estándar de Python es que son "flexibles". Pueden contener un número, luego un texto, luego otra lista. Esa flexibilidad consume mucha memoria y tiempo de procesamiento.

NumPy resuelve esto introduciendo el **ndarray** (n-dimensional array). Un array de NumPy obliga a que todos sus elementos sean del mismo tipo (generalmente números). Al saber que "todo son números", la computadora puede organizar la memoria de forma contigua y realizar cálculos masivos en un solo paso.

El concepto clave: Vectorización En lugar de escribir un bucle `for` para sumar 1 a cada uno de un millón de números, en NumPy simplemente haces `array + 1`. Esto se llama **vectorización**. Es más rápido de escribir, más fácil de leer y miles de veces más eficiente para la computadora.

2. La Mesa de Trabajo: Pandas

Si NumPy es el motor, **Pandas** es el tablero de control y la carrocería del vehículo. Construida directamente sobre NumPy, Pandas está diseñada para trabajar con **datos tabulares** (datos que parecen tablas de Excel o SQL).

¿Por qué existe Pandas?

Rara vez los datos del mundo real son solo "matrices de números". Los datos reales tienen:

- **Nombres de columnas** (ej. "Nombre", "Precio", "Fecha").
- **Tipos mixtos** (una columna de texto, otra de números, otra de fechas).
- **Valores faltantes** (celdas vacías porque alguien olvidó llenar una encuesta).
- **Índices** (formas de identificar una fila, como un ID de cliente).

Pandas toma la eficiencia de NumPy y le añade una capa de "etiquetas" o nombres. Esto nos permite hablarle a los datos en términos humanos: *"Dame la media de la columna 'Salario' para los empleados del departamento 'Ventas'"*.

3. Comparativa: ¿Cuándo usar cuál?

Aunque trabajan juntas, cumplen roles distintos. Una regla de oro pedagógica es: **"NumPy trabaja con números; Pandas trabaja con datos"**.

Característica	NumPy	Pandas
Estructura principal	ndarray (Matriz)	DataFrame (Tabla)
Tipo de datos	Homogéneo (todo números)	Heterogéneo (mezcla tipos)
Identificación	Solo por posición (índice 0, 1, 2...)	Por posición y por nombre (etiquetas)
Uso ideal	Álgebra lineal, procesamiento de imágenes, cálculos pesados.	Análisis exploratorio, limpieza de datos, leer archivos CSV/Excel.

Característica	NumPy	Pandas
Celdas vacías	Difíciles de manejar.	Diseñado para detectar y limpiar nulos.

Un ejemplo del mundo real: El Inventario de una Tienda

Imagina que representas el stock de una tienda:

1. **Con NumPy:** Tendrías una matriz como `[[10, 5.50], [20, 12.00]]`. Sabes que la primera columna es cantidad y la segunda es precio, pero solo porque tú lo memorizaste. Si alguien más ve el código, no tiene idea de qué significan esos números.
2. **Con Pandas:** Tendrías una tabla (DataFrame) con encabezados:
 - **Producto:** ["Camisa", "Pantalón"]
 - **Cantidad:** [10, 20]
 - **Precio:** [5.50, 12.00]

En Pandas, puedes escribir `df['Precio'].mean()` para saber el precio promedio. Es intuitivo y profesional.

4. La Relación de Dependencia

Es vital entender que **Pandas no reemplaza a NumPy**, sino que lo extiende. De hecho, cada columna de un DataFrame de Pandas es, en su interior, un array de NumPy con una "etiqueta" pegada encima. Cuando realizas una operación en Pandas, como sumar dos columnas, Pandas le pide a NumPy que haga el cálculo matemático rápido en el fondo y luego vuelve a ponerle las etiquetas a los resultados para mostrártelos. Por eso, para ser un experto en Ciencia de Datos, necesitas entender ambos: NumPy para entender cómo se procesan los datos y Pandas para saber cómo manipularlos.

5. Terminología Esencial (El Diccionario del Data Scientist)

Para hablar el lenguaje de los datos, debemos dominar estos términos:

- **Array:** Un contenedor de elementos organizados en una o más dimensiones (como una lista o una cuadrícula).
- **Vector:** Un array de una sola dimensión (una fila o una columna de números).
- **Matriz:** Un array de dos dimensiones (filas y columnas).
- **Series:** El objeto fundamental de Pandas para una sola columna de datos.
- **DataFrame:** El objeto fundamental de Pandas para una tabla completa (muchas Series juntas).
- **Índice (Index):** La etiqueta que identifica a cada fila. Por defecto es 0, 1, 2..., pero puede ser una fecha o un nombre.
- **Shape:** Un atributo que te dice cuántas filas y columnas tiene tu estructura. Ejemplo: `(100, 5)` significa 100 filas y 5 columnas.
- **NaN (Not a Number):** La forma en que NumPy y Pandas representan un valor faltante o nulo.

6. Errores Comunes y Confusiones

Como principiante, es muy probable que te enfrentes a estas trampas conceptuales:

Confusión 1: Pensar que son herramientas separadas Muchos estudiantes preguntan: "¿Aprendo NumPy o aprendo Pandas?". La respuesta es que en el 90% de los proyectos usarás ambos. Importarás Pandas para leer tu archivo CSV y usarás funciones de NumPy para realizar operaciones matemáticas avanzadas o transformaciones rápidas en esas columnas.

Confusión 2: El error de la "Copia vs. Vista" Un error muy frustrante ocurre al intentar cambiar un dato en Pandas y ver que el cambio no se guardó. Pandas a veces te muestra una "vista" (un espejo) de los datos y no los datos originales por razones de eficiencia. Siempre verifica si tu operación modificó el DataFrame original.

Confusión 3: Los "datos sucios" Muchos principiantes asumen que Pandas "arregla" los datos automáticamente al cargarlos. Si tienes una columna que debería ser de números pero tiene un solo texto (ej: "Agotado"), Pandas podría convertir TODA la columna a texto, impidiéndote hacer cálculos. La inspección inicial es clave.

7. Síntesis y Próximos Pasos

Hemos establecido las bases de nuestro arsenal de herramientas:

- **NumPy** es nuestra calculadora de alta potencia, optimizada para la velocidad y la eficiencia mediante la vectorización.
- **Pandas** es nuestro centro de gestión de datos, que nos permite organizar, etiquetar y limpiar información compleja.

Entender la relación entre ambos es como entender la relación entre los ladrillos (NumPy) y el edificio completo (Pandas). Los ladrillos son la unidad fundamental de fuerza, pero el edificio es donde realmente ocurre la actividad humana y el análisis.

En las siguientes unidades, profundizaremos en cómo crear estas estructuras (Series y DataFrames) y cómo empezar a interrogar a nuestros datos para obtener respuestas de negocio reales. Pero recuerda siempre: detrás de cada tabla hermosa de Pandas, hay un ejército de arrays de NumPy trabajando a toda velocidad.

Introducción a Pandas: Series y DataFrames

Introducción a Pandas: Series y DataFrames

¡Bienvenidos a una de las unidades más importantes en tu camino como Data Scientist! Si hasta ahora has sentido que trabajar con listas y diccionarios en Python es un poco como organizar una biblioteca sin estantes, **Pandas es el sistema de estanterías profesional que estabas esperando.**

En el mundo real, los datos no vienen en simples filas de números. Vienen en tablas con nombres de clientes, fechas de ventas, categorías de productos y precios. Pandas es la librería de Python diseñada específicamente para manejar estos datos "etiquetados" y "tabulares" con una facilidad asombrosa. Si sabes usar una hoja de cálculo (como Excel o Google Sheets), ya tienes la mitad del camino recorrido para entender Pandas.

En esta unidad, aprenderemos cómo Pandas organiza la información en dos estructuras fundamentales: las **Series** (columnas individuales) y los **DataFrames** (tablas completas).

1. ¿Qué es Pandas y por qué es el estándar de la industria?

Pandas es una librería de código abierto que proporciona estructuras de datos de alto rendimiento y herramientas de análisis fáciles de usar. Su nombre proviene de "*Panel Data*" (un término econométrico para conjuntos de datos que incluyen observaciones sobre múltiples períodos de tiempo).

El problema que resuelve

Antes de Pandas, los programadores de Python tenían que usar listas de diccionarios o arrays de NumPy para manejar tablas. Esto era problemático porque:

1. **Falta de etiquetas:** En NumPy, si quieres la columna de "Precios", debes recordar que es la columna índice 2. En Pandas, simplemente pides "Precios".
2. **Tipos de datos mixtos:** NumPy prefiere que todos los datos sean del mismo tipo (ej. todos números). Pandas permite que una columna sea texto (Nombres) y otra sea número (Edad), conviviendo en la misma tabla perfectamente.
3. **Datos faltantes:** Pandas tiene herramientas integradas para manejar esos molestos "huecos" o valores nulos que siempre aparecen en la vida real.

En resumen: Pandas convierte a Python en una herramienta de análisis de datos tan potente (o más) que SQL o Excel, pero con toda la flexibilidad de un lenguaje de programación.

2. La Serie: El bloque de construcción unidimensional

Imagina que tienes una lista de los precios de 5 productos. En Python básico, verías esto: `[10.5, 22.0, 5.0, 15.8, 30.0]`. Pero, ¿cuál es cuál?

Una **Serie** de Pandas es como esa lista, pero **con esteroides**. Es una estructura unidimensional que contiene un array de datos y un array asociado de etiquetas, llamado **índice**.

Anatomía de una Serie

- **Valores:** Los datos reales (los números o textos).
- **Índice (Index):** Las etiquetas que identifican a cada valor. Si no asignas uno, Pandas pondrá números (0, 1, 2...).

Ejemplo concreto:

Si creamos una Serie para representar el stock de frutas:

Índice (Etiqueta)	Valor (Dato)
Manzanas	50
Bananas	120
Cerezas	30

Aquí, "Manzanas" es la etiqueta que te permite acceder al valor 50 de forma inmediata, sin tener que saber en qué posición está.

3. El DataFrame: La tabla bidimensional

Si la Serie es una columna, el **DataFrame** es la hoja de cálculo completa. Es la estructura más utilizada en Ciencia de Datos.

Un DataFrame es una estructura de datos bidimensional (filas y columnas) orientada a etiquetas. Puedes imaginarlo como un **diccionario de Series**, donde todas comparten el mismo índice.

Componentes clave del DataFrame

1. **Datos (Data):** La información organizada en celdas.
2. **Índice de filas (Index):** Las etiquetas de las filas (quién es cada registro).
3. **Columnas (Columns):** Los nombres de cada columna (qué tipo de información guardamos).

Ejemplo en el mundo real:

Imagina un dataset de una tienda:

	Producto	Precio	Cantidad
0	Camiseta	15.0	100
1	Pantalón	30.0	50
2	Zapatos	60.0	25

- **¿Cuál es el índice?** Los números 0, 1, 2 a la izquierda.
- **¿Cuáles son las columnas?** "Producto", "Precio" y "Cantidad".

4. Creación de Series y DataFrames

En Pandas, no siempre cargamos archivos externos; a menudo necesitamos crear estas estructuras a partir de otros objetos de Python que ya conocemos, como **listas** o **diccionarios**.

Desde Listas

Es la forma más sencilla. Creas una lista y se la pasas a Pandas.

python

```
import pandas as pd
# Crear una Serie
precios = pd.Series([10.5, 22.0, 5.0],
name="Precio_Productos")
print(precios)
```

Desde Diccionarios (El método más común)

Los diccionarios son perfectos para crear DataFrames porque sus "llaves" se convierten naturalmente en los nombres de las columnas.

```
python
# Crear un DataFrame desde un diccionario
data_dict = { 'Estudiante':
['Ana', 'Luis', 'Marta'], 'Nota': [9.5, 8.0, 10.0], 'Asistencia':
[True, True, False] }
df_estudiantes = pd.DataFrame(data_dict)
```

¿Por qué es genial este método? Porque Pandas alinea automáticamente los datos. Sabe que 'Ana' tiene un 9.5 y que su asistencia es True porque están en la misma posición de sus respectivas listas.

5. Relación entre datos tabulares y Análisis Exploratorio (EDA)

Cuando recibes un dataset por primera vez, no empiezas a aplicar modelos matemáticos complejos de inmediato. Primero haces **EDA (Exploratory Data Analysis)**.

Pandas es tu linterna en la oscuridad del EDA. Gracias a que los DataFrames están etiquetados, puedes hacer preguntas de negocio rápidamente:

- "¿Cuál es la media de la columna 'Ventas'?"
- "¿Cuántos valores nulos hay en la columna 'Email'?"
- "¿Qué productos tienen un 'Stock' menor a 10?"

Sin las etiquetas de columnas e índices, tendrías que iterar con bucles `for` sobre listas anidadas, lo cual es lento y propenso a errores. Pandas hace que estas preguntas se respondan en una sola línea de código.

6. Errores comunes y mejores prácticas

El error de la dimensión

Un error típico de principiante es intentar usar una función de DataFrame en una Serie o viceversa.

- **Recuerda:** Si pides una sola columna de un DataFrame (`df['columna']`), Pandas te devuelve una **Serie**. Si pides varias (`df[['col1', 'col2']]`), te devuelve un **DataFrame**.

El índice no es solo un número

Muchos estudiantes olvidan que el índice puede ser texto. Por ejemplo, en un dataset de países, el índice podría ser el nombre del país (Argentina, México, España) en lugar de 0, 1, 2. Esto facilita mucho la búsqueda de datos específicos.

Tipos de datos heterogéneos

A diferencia de NumPy, donde si mezclas un número con un texto todo se convierte a texto, Pandas es más flexible. Sin embargo, **asegúrate siempre de revisar tus tipos**. Si una columna de precios se carga como texto ("10"), no podrás hacer cálculos matemáticos con ella hasta que la limpies.

7. Síntesis y Conexiones

Hemos pasado de los arrays matemáticos de NumPy a las estructuras organizadas y etiquetadas de Pandas.

1. La **Serie** es la unidad básica (1D), ideal para una sola variable.
2. El **DataFrame** es la tabla reina (2D), donde ocurre la magia del análisis.

3. Ambas usan **índices** para que no tengamos que adivinar dónde están los datos. En las siguientes unidades, aprenderás a "interrogar" a estos DataFrames: cómo pedirles solo las filas que cumplen una condición o cómo obtener estadísticas rápidas de miles de registros en un segundo. ¡Prepárate, porque aquí es donde los datos empiezan a contar historias!

Para practicar (No entregable):

Título: Construye tu primer cuaderno con Series y DataFrames

Propósito del ejercicio

Este ejercicio tiene como objetivo que pongas en práctica la creación manual de estructuras de datos en Pandas y comprendas cómo se organizan internamente mediante etiquetas e índices, antes de pasar a datasets masivos.

Pasos sugeridos

1. **Configuración inicial:** Crea un repositorio en GitHub llamado `introduccion_pandas_ejercicio`. Clónalo en tu entorno local y abre un Jupyter Notebook, o nombrá un cuaderno de Colab con el mismo nombre.
2. **Creación de una Serie:**
 - Imagina que estás registrando la temperatura máxima de los últimos 7 días.
 - Crea una Serie de Pandas donde los **valores** sean las temperaturas y el **índice** sean los nombres de los días (Lunes, Martes, etc.).
 - Agrega una nota en una celda Markdown explicando por qué en este caso es mejor usar una Serie con etiquetas que una simple lista de Python.
3. **Construcción de un DataFrame:**
 - Diseña un diccionario que contenga información de 5 productos de una tienda ficticia. Debe tener al menos 3 columnas: `Producto` (texto), `Precio` (decimal) y `En_Stock` (booleano).
 - Convierte ese diccionario en un DataFrame de Pandas.
4. **Inspección técnica:**
 - Usa los atributos `.index`, `.columns` y `.dtypes` para visualizar la estructura de tu DataFrame.
 - Observa cómo Pandas asignó automáticamente los tipos de datos a cada columna.
5. **Carga de datos (Opcional pero recomendado):**
 - Crea un archivo pequeño llamado `datos_prueba.csv` con un editor de texto (o desde Python) con solo 3 filas de datos.
 - Usa `pd.read_csv()` para cargarlo y compáralo con el DataFrame que creaste manualmente.

Errores comunes a evitar

- **Olvidar importar pandas:** Recuerda siempre empezar con `import pandas as pd`.
- **Desigualdad de longitud:** Al crear un DataFrame desde un diccionario, todas las listas deben tener exactamente la misma cantidad de elementos. Si una tiene 5 y otra 4, Python lanzará un error.

Preprocesamiento de Datos

Preprocesamiento de Datos — Estrategias de Eliminación e Imputación de Valores Ausentes

1. Introducción Teórica: El Fenómeno de los Datos Ausentes

En entornos productivos de Data Science, la presencia de datos faltantes (identificados comúnmente como NA o NaN — *Not a Number*) es una constante inevitable. Estos vacíos de información se originan por fallas en los sensores de captura, interrupciones en los pipelines de integración de software, o registros incompletos en las bases de datos transaccionales.

El tratamiento de estos valores es una fase crítica del preprocesamiento. Un manejo incorrecto introduce sesgos en las distribuciones estadísticas y degrada la capacidad predictiva de los modelos de Machine Learning, los cuales, en su mayoría, no admiten registros nulos en sus matrices de entrada.

2. Estrategias de Mitigación: Eliminación vs. Imputación

Existen dos enfoques fundamentales para abordar la pérdida de datos en un dataset tabular:

A. Eliminación (Deletion)

Consiste en descartar filas o columnas que contienen registros nulos.

- **Criterio técnico:** Se aplica cuando la densidad de datos ausentes en un registro es tan alta que intentar recuperarlo distorsionaría la realidad del fenómeno observado.

B. Imputación (Imputation)

Consiste en reemplazar el valor ausente por un estimador estadístico calculado a partir de los datos disponibles.

- **Criterio técnico:** Se utiliza cuando la pérdida de información es marginal, permitiendo preservar el tamaño del dataset sin alterar significativamente las propiedades estadísticas de la variable.

Cuadro Comparativo de Impacto Estadístico

Estrategia	Ventaja Principal	Desventaja / Riesgo	Impacto en la Varianza
Eliminación por Filas	No inventa datos; mantiene la certeza de los registros restantes. Preserva el tamaño de la muestra y es computacionalmente eficiente.	Reduce el tamaño de la muestra y puede sesgarla si los datos no faltan al azar.	Puede aumentar el error estándar por reducción de N .
Imputación por la Media		Subestima la variabilidad de la columna y altera las correlaciones.	Reduce la varianza de la variable de forma artificial.

3. El Algoritmo de Decisión Basado en Reglas

Para estandarizar el tratamiento de un dataset comercial, aplicaremos un flujo lógico estricto basado en la **densidad de ausencia por registro (fila)**.

Evaluaremos cada fila i en una matriz de dimensiones $M \times N$:

Si (Cantidad de NA en fila i) $> 1 \Rightarrow$ Eliminar registro completo
Si (Cantidad de NA en fila i) $= 1 \Rightarrow$ Imputar mediante la Media de la columna

El Rigor de la Línea de Base: Cálculo de Medias

Un principio metodológico fundamental en la ingeniería de datos es **evitar el Data Leakage (fuga de datos) y el sesgo de selección**. Por lo tanto:

Regla de Oro: Las medias poblacionales de cada columna deben calcularse **antes de realizar cualquier eliminación de filas**, considerando única y exclusivamente los valores numéricos presentes.

Si elimináramos filas antes de calcular la media, estaríamos alterando la distribución original de la variable, invalidando la representatividad del valor imputado.

Integración, Agregación y Preprocesamiento Avanzado de Datos

Integración, Agregación y Preprocesamiento Avanzado de Datos

1. Combinación de Datos: Operaciones Relacionales en Pandas

En la práctica profesional de la Ciencia de Datos, la información se encuentra distribuida de forma normalizada en múltiples fuentes (tablas relacionales). La combinación de conjuntos de datos consiste en unificar estas fuentes mediante columnas indexadas compartidas llamadas **claves (keys)**.

Requisitos Técnicos de las Claves

- **Compatibilidad de Tipos:** Las claves de ambas tablas deben compartir exactamente el mismo tipo de dato en memoria (ej. `int64` con `int64`). Contrastar un entero `1` con una cadena de texto `'1'` impedirá la coincidencia sintáctica.
- **Homogeneidad:** Valores vacíos o codificaciones inconsistentes deben corregirse antes de la fusión para evitar emparejamientos espurios o pérdida de registros.

Tipos de Uniones (Joins) y su Comportamiento Setorial

La forma en que se preservan o descartan las filas sin coincidencia define el tipo de unión:

Tipo de Join	Comportamiento Computacional	Impacto en el Dataset Resultante
Inner Join	Conserva únicamente las filas cuyas claves están presentes en ambos DataFrames (Intersección).	Descarta registros no duplicados en ambas fuentes.
Left Join	Preserva la totalidad de las filas del DataFrame izquierdo y añade las columnas del derecho donde haya coincidencia.	Si no hay coincidencia, rellena con <code>NaN</code> . Ideal para enriquecer una tabla base.
Right Join	Simétrico al Left Join. Preserva el DataFrame derecho.	Menos utilizado en pipelines lineales; se prefiere reordenar los argumentos en un Left Join.
Outer Join	Conserva absolutamente todas las filas de ambos DataFrames (Unión).	Genera estructuras dispersas con alta presencia de <code>NaN</code> donde las claves no coinciden.

Buenas Prácticas y Control de Calidad en `pd.merge()`

La fusión de datos puede alterar la granularidad del dataset de forma silenciosa. Si una clave está duplicada en una de las tablas, la operación ejecutará un **producto cartesiano** para esas coincidencias, multiplicando inesperadamente el número de filas.

Para mitigar esto, la librería Pandas provee mecanismos de validación defensiva:

Python

```
import pandas as pd
# Fusión defensiva con validación de cardinalidad de relaciones
df_resultado = pd.merge(left=orders, right=customers[['customer_id', 'customer_name']],
                        on='customer_id', how='left', validate='many_to_one',
                        # Lanza un ValidationError si la clave en 'right' no es única
                        suffixes=('_order', '_customer'))
# Evita colisiones de nombres de columnas
```

🔍 **Herramienta de Auditoría:** Pasar el argumento `indicator=True` creará una columna especial llamada `_merge` que etiqueta el origen de cada fila (`left_only`, `right_only`, `both`), permitiendo auditar pérdidas de información post-merge.

2. Agregación de Datos: El Paradigma Split-Apply-Combine

Para resumir la información y extraer métricas de negocio, utilizamos la operación de agrupamiento. Computacionalmente, este proceso se rige bajo el patrón arquitectónico **Split-Apply-Combine** (Dividir-Aplicar-Combinar).

1. **Split (Dividir):** El conjunto de datos original se particiona en sub-tablas lógicas basadas en los valores únicos de una o más variables categóricas.
2. **Apply (Aplicar):** Se ejecuta una función matemática o de transformación sobre cada uno de los subgrupos de forma aislada.
3. **Combine (Combinar):** Se colapsan los resultados parciales y se consolidan en una nueva estructura matricial unificada.

Reducción de Dimensionalidad (`agg`) vs. Proyección Estructural

(`transform`)

Es fundamental diferenciar el impacto estructural de estas operaciones al procesar datos:

- **Método `.agg()` (Aggregation):** Reduce las observaciones. Si un grupo tiene 1,000 filas, `.agg()` devuelve una sola fila con el estadístico descriptivo computado (suma, media, varianza).
- **Método `.transform()` (Transformation):** Preserva la dimensionalidad. Devuelve un objeto del mismo tamaño exacto que el DataFrame original. Aplica el cálculo por grupo pero proyecta o transmite (*broadcasts*) el resultado a cada fila original, lo cual es indispensable para operaciones de normalización intragrupo.

Python

```
# Uso de .agg() para reportería y reducción de dimensionesdf_resumen = df.groupby(['month', 'category']).agg(    total_sales=('sales', 'sum'),    avg_price=('price', 'mean'),    n_orders=('order_id', 'nunique')).reset_index() # Evita índices jerárquicos complejos
# Uso de .transform() para normalización intragrupo (Z-score por categoría)df['sales_z_category'] = df.groupby('category')['sales'].transform(    lambda x: (x - x.mean()) / x.std())
```

3. Diagnóstico y Análisis Exploratorio de Datos (EDA) Avanzado

Ninguna transformación o modelo predictivo es válido si no se comprende rigurosamente la calidad de la base de datos de origen. El EDA estructurado permite auditar la consistencia estadística y de software de los datos.

Identificación de Anomalías y Datos Ausentes Ocultos

En múltiples sistemas comerciales, los valores faltantes no se codifican explícitamente como nulos (NaN), sino mediante valores centinela (como -1, cadenas vacías "" o texto 'NA'). Estos valores alteran drásticamente los estadísticos descriptivos si no se sanean al inicio del pipeline.

Diagnóstico Visual Profesional

- **Histogramas:** Permiten diagnosticar el sesgo (*skewness*) y la presencia de colas largas en variables financieras (como ventas o ingresos), determinando si la distribución requiere una transformación logarítmica.
- **Boxplots:** Revelan visualmente la dispersión, la simetría de los datos y la ubicación geométrica de observaciones atípicas basadas en el Rango Intercuartílico.
- **Heatmaps de Correlación:** Evalúan el grado de asociación lineal matemática entre variables numéricas, permitiendo anticipar problemas de **multicolinealidad** en modelos lineales.

4. Técnicas Sólidas de Limpieza y Tratamiento de Outliers

Imputación Estadística por la Media

Consiste en reemplazar los valores faltantes confirmados (NaN) por la media aritmética de la columna, utilizando exclusivamente las observaciones presentes.

⚠ **Impacto Estadístico:** La imputación por la media introduce un sesgo artificial: **subestima la varianza original** de la variable y altera las correlaciones con otras variables. Debe emplearse con pleno conocimiento de esta contracción en la dispersión de los datos.

Atenuación de Outliers mediante Winsorización

En lugar de eliminar registros extremos (lo que reduce el tamaño muestral y podría eliminar información valiosa del negocio), se aplica la técnica estadística de **Winsorización**.

Esta técnica limita los valores extremos asignándoles un tope correspondiente a un percentil predeterminado en los extremos de la distribución (típicamente el Percentil 1 para el límite inferior y el Percentil 99 para el límite superior). De esta forma, cualquier valor extremo se iguala, estabilizando la varianza del dataset sin alterar la cantidad de registros.

5. Teorema de Escalamiento y Normalización de Variables

Muchos algoritmos de Machine Learning (como regresiones regularizadas, KNN o redes neuronales) son sensibles a las escalas de las variables. Si una columna posee valores en miles de millones y otra en decimales, los algoritmos ponderarán erróneamente la variable de mayor magnitud.

Para homogeneizar los rangos de las variables numéricas se aplican dos metodologías:

A. Estandarización Clásica (Z-Score)

Transforma la variable para que posea una media igual a 0 y una desviación estándar igual a 1.

- **Restricción:** Es altamente sensible a la presencia de outliers extremos, ya que tanto la media como la desviación estándar se ven severamente afectadas por valores atípicos.

B. Escalamiento Robusto (Robust Scaling)

Utiliza estadísticos de orden o posición, haciéndolo inmune a la distorsión provocada por los valores atípicos extremos.

Donde el Rango Intercuartílico (IQR) se define como la diferencia entre el tercer y el primer cuartil:

$$\{\text{IQR}\} = P_{\{75\}} - P_{\{25\}}$$

Este método es el estándar industrial cuando las variables presentan distribuciones con colas largas o asimetría pronunciada.

6. Arquitectura de Pipelines, Reproducibilidad y Versionado de Artefactos

Un flujo de trabajo en Ciencia de Datos debe ser **determinista y reproducible**. Para garantizar que el procesamiento arroje los mismos resultados exactos en cualquier entorno productivo, se deben implementar las siguientes directrices:

- **Control de Aleatoriedad:** Fijar la semilla pseudoaleatoria (`random seed`) en cualquier función que involucre partición o muestreo de datos.
- **Aislamiento de Artefactos:** Guardar de forma independiente los estados físicos del dataset (Dataset Limpio, Dataset Estandarizado con Z-score, Dataset con Escalamiento Robusto). Esto facilita la auditoría del pipeline y permite alimentar diferentes modelos según sus requerimientos matemáticos.

- **Registro de Ejecución (Logging):** Almacenar metadatos del procesamiento (fecha de ejecución, número de filas procesadas, vector de medias utilizadas para la imputación). Esto dota al pipeline de trazabilidad y permite monitorear desviaciones en los datos (*data drift*) a lo largo del tiempo.

La Sinergia de Datos: NumPy y Pandas en Profundidad

NumPy y Pandas: El Motor y la Carrocería de la Ciencia de Datos

Imagina que decides abrir una tienda de café de especialidad. Durante el primer mes, anotas cada venta en una libreta: el tipo de grano, el precio, la hora y si el cliente pidió leche vegetal. Al final del mes, tienes una montaña de datos. ¿Cómo podrías calcular rápidamente cuánto dinero ganaste? ¿Cómo podrías saber qué hora es la más concurrida o cuál es el producto estrella?

Si intentas hacer esto con papel y lápiz, tardarás horas. Si usas Python básico (listas y bucles `for`), será más rápido, pero el código se volverá complejo y lento a medida que tu negocio crezca. Aquí es donde entran los dos pilares fundamentales del ecosistema de datos en Python: **NumPy y Pandas**.

En esta unidad, vamos a profundizar en la relación simbiótica entre estas dos bibliotecas. Entenderemos por qué no son competencia, sino aliadas, y cómo cada una resuelve una parte crítica del rompecabezas de la Ciencia de Datos.

1. NumPy: El Motor Numérico de Alta Precisión

Para entender NumPy, debemos entender el concepto de **homogeneidad**.

Imagina una fila de asientos en un estadio de fútbol. Todos los asientos son idénticos, están a la misma distancia y tienen el mismo tamaño. Debido a esta estructura perfecta, el personal de limpieza sabe exactamente dónde termina un asiento y empieza el siguiente sin tener que mirar. Esto es un **array de NumPy**.

El Problema que Resuelve NumPy

Las listas nativas de Python son como una "bolsa de sorpresas": pueden contener un número, luego una palabra, luego otra lista. Esta flexibilidad es genial para programar aplicaciones generales, pero es un desastre para el rendimiento numérico. Python tiene que "preguntarle" a cada elemento: "¿Qué eres? ¿Eres un número? ¿Puedo sumarte?".

NumPy elimina esa pregunta. Al exigir que todos los elementos sean del mismo tipo (habitualmente números), NumPy puede hablarle directamente al procesador de tu computadora.

Conceptos Clave

- **Vectorización:** Es la capacidad de realizar una operación en todo un bloque de datos a la vez. Si tienes 1,000,000 de precios y quieres aplicar un 10% de descuento, en Python tradicional harías un bucle. En NumPy, simplemente haces `precios * 0.9`. Es como si, en lugar de pintar cada asiento del estadio uno por uno, tuvieras una máquina gigante que stampa pintura en toda la fila en un segundo.
 - **dtype (Data Type):** Es el "pasaporte" de los datos en un array. Define si estamos trabajando con enteros, decimales (floats) o booleanos. Saber el `dtype` es vital porque determina cuánta memoria ocupa cada número.
-

2. Pandas: La Carrocería para Datos del Mundo Real

Si NumPy es el motor potente que hace girar las ruedas, **Pandas es el tablero, los asientos de cuero y el GPS**.

NumPy es excelente para matrices matemáticas, pero los datos del mundo real no son solo números; tienen etiquetas. Los datos de tu cafetería tienen nombres de columnas ("Precio", "Fecha", "Tipo de Café") y etiquetas de filas (el número de ticket o la hora).

La Analogía: De la Rejilla a la Hoja de Cálculo

Una forma muy efectiva de diferenciarlos es:

- **NumPy** es una rejilla matemática pura.
- **Pandas** es una hoja de cálculo (como Excel), pero programable y con superpoderes.

Las dos estructuras reina de Pandas

1. **Series:** Imagina una sola columna de tu hoja de cálculo. Tiene un nombre y un índice (una etiqueta para cada celda). Es unidimensional.
2. **DataFrame:** Es la tabla completa. Es una colección de Series que comparten el mismo índice. Es la estructura que usarás el 90% del tiempo en Ciencia de Datos.
Importante: Un DataFrame de Pandas está construido *sobre* arrays de NumPy. Cada columna de un DataFrame es, en esencia, un array de NumPy con una etiqueta encima. Por eso Pandas es tan rápido: hereda el motor de alto rendimiento de NumPy pero le añade una capa de usabilidad humana.

3. ¿Por qué necesitamos ambos? (La Relación Simbiótica)

Un error común entre principiantes es pensar: *"Si Pandas es más fácil y usa NumPy por debajo, ¿por qué no usar solo Pandas?"*.

La respuesta reside en la especialización:

Característica	NumPy	Pandas
Tipo de Datos	Homogéneo (todo números)	Heterogéneo (números, texto, fechas)
Uso Principal	Cálculos matemáticos, álgebra lineal	Manipulación de tablas, limpieza de datos
Estructura	Arrays multidimensionales (n-d)	Series y DataFrames (tabulares)
Manejo de Nulos	No es su fuerte	Experto en detectar y tratar datos faltantes

Caso de uso real: Imagina que estás trabajando en una aplicación financiera. Usarías **Pandas** para leer un archivo CSV con miles de transacciones, filtrar las que ocurrieron en diciembre y agruparlas por categorías. Una vez que tienes los datos limpios y organizados, podrías pasar esos valores a **NumPy** para realizar simulaciones matemáticas complejas o cálculos estadísticos de alto rendimiento.

4. Aplicaciones en la Industria

Para aterrizar estos conceptos, veamos cómo se utilizan en sectores específicos:

Finanzas

Pandas nació originalmente en el sector financiero (AQR Capital Management). Se usa para gestionar **Series Temporales**. Si tienes el precio de una acción cada segundo durante cinco años, Pandas te permite calcular "medias móviles" o encontrar el día de mayor volatilidad con apenas dos líneas de código.

Análisis de Ventas (Retail)

Las empresas de retail usan Pandas para unir tablas de diferentes fuentes. Por ejemplo, unir la tabla de "Ventas" con la de "Inventario" usando el ID del producto como índice común. Esto permite detectar faltantes de stock antes de que ocurran.

Investigación Científica

NumPy es el estándar de facto. Desde procesar imágenes de telescopios (que se ven como matrices de píxeles) hasta analizar señales sísmicas, NumPy permite realizar cálculos que en Python puro tardarían días, en cuestión de segundos.

5. Errores Comunes y Mitos

El Mito del Bucle `for`

Error: Intentar recorrer un DataFrame fila por fila con un bucle para cambiar valores. **Por qué es un problema:** Esto es extremadamente lento. Destruye la ventaja competitiva de NumPy y Pandas. **La forma correcta:** Utiliza **operaciones vectorizadas**. Si necesitas duplicar el valor de una columna, haz `df['columna'] * 2` en lugar de iterar.

Confundir NumPy con Pandas

Error: Intentar poner nombres de columnas a un array de NumPy. **La forma correcta:** Si necesitas etiquetas (nombres), tu estructura es un DataFrame de Pandas. Si solo necesitas hacer álgebra de matrices pura con números, quédate con NumPy.

"Los datos ya vienen limpios"

Error: Olvidar revisar los datos faltantes (NaN). **Realidad:** En el mundo real, los sensores fallan, los usuarios se saltan preguntas de encuestas y los sistemas pierden registros. Pandas trata los valores faltantes como ciudadanos de primera clase, dándote herramientas para eliminarlos o rellenarlos con inteligencia (por ejemplo, con el promedio).

6. Síntesis y Modelo Mental

Para cerrar esta unidad, construye este modelo mental en tu cabeza:

1. **NumPy es la fundación:** Eficiencia, velocidad, números y homogeneidad. Piensa en él como el bloque de construcción básico, denso y resistente.
 2. **Pandas es la interfaz:** Flexibilidad, etiquetas, datos reales y heterogeneidad. Piensa en él como el organizador que toma esos bloques y les pone etiquetas para que sepamos qué estamos construyendo.
- ¿Qué sigue?** Ahora que entiendes la profundidad teórica de estas herramientas, estamos listos para pasar a la acción. En las próximas unidades, aprenderás a usar las funciones de "inspección inicial". Estas funciones (`head`, `info`, `describe`) son tus ojos: te permitirán asomarte a un dataset gigantesco y entender su salud y estructura en menos de cinco segundos.

Sin este motor (NumPy) y esta carrocería (Pandas), la Ciencia de Datos moderna simplemente no sería posible en Python. Todo lo que construyas de aquí en adelante, desde gráficos estadísticos hasta modelos de Inteligencia Artificial, se apoyará en esta base.

¡Prepárate para empezar a explorar tus propios datasets!

Inspección inicial de datos con `head`, `info` y `describe`

Inspección inicial de datos con `head`, `info` y `describe`

Tarjeta 1 de 8

Aprende a realizar un perfilado inicial de tus datasets utilizando funciones clave de Pandas para entender su estructura y calidad antes del análisis.

Inspección inicial de datos: Conociendo tu estructura

Cargar un conjunto de datos es solo el primer paso. Antes de realizar cualquier cálculo complejo o visualización, un científico de datos debe "**tomar el pulso**" a la información. Este proceso se conoce como inspección inicial, y en Pandas disponemos de tres pilares fundamentales para ello: `head()`, `info()` y `describe()`.

¿Por qué inspeccionar antes de analizar?

Imagina que recibes una caja cerrada que se supone contiene herramientas. Antes de empezar a construir una casa, abres la caja, verificas qué herramientas hay, si alguna está rota o si falta lo esencial. En Data Science, inspeccionar nos permite:

1. **Validar la carga:** ¿Se importó el archivo correctamente con sus columnas en orden?
2. **Entender la estructura:** ¿Qué tipos de datos tenemos (números, texto, fechas)?
3. **Detectar problemas:** ¿Hay "agujeros" (valores nulos) o datos inconsistentes?

Uso efectivo de las tarjetas

Estas tarjetas de repaso están diseñadas para ayudarte a diferenciar qué herramienta usar según la pregunta que quieras responder sobre tus datos.

- **Recomendación:** Intenta responder mentalmente o en voz alta el concepto de la tarjeta antes de revelarla. Si puedes explicar el *propósito* además del nombre de la función, habrás dominado el concepto.

Pregunta

¿Qué es `df.head()` y cuál es su utilidad principal?

Pre-entrega: Estructura inicial del dataset del proyecto

Actividad

Pre-entrega: Estructura inicial del dataset del proyecto

¡Felicidades! Has completado el módulo de **NumPy** y **Pandas**, las dos herramientas pilares sobre las cuales se construye prácticamente todo el ecosistema de Ciencia de Datos en Python. Hasta ahora, has aprendido a pensar en términos de arrays, a manipular tablas con el poder de los DataFrames y a interrogar a tus datos usando filtros y resúmenes estadísticos.

Este checkpoint es un momento clave en tu formación. Es el punto donde dejas de trabajar con datasets de ejemplo y comienzas a darle forma a tu propio **Proyecto Final**.

Contexto y posición en la cadena

En el módulo de "Nivelación" y "Fundamentos de Python", preparaste tu entorno de trabajo y aprendiste la lógica de programación. Ahora, en este primer checkpoint, aplicarás lo aprendido sobre Pandas y NumPy para "darle una identidad" a tu proyecto.

Lo que construyas hoy no es el análisis definitivo, sino la base sólida (saneada y explorada) sobre la cual aplicarás técnicas de visualización en el próximo módulo y, eventualmente, modelos de Machine Learning al finalizar el programa.

Objetivo de aprendizaje

Consolidar la capacidad de cargar un dataset real, inspeccionar su estructura interna, validar tipos de datos y realizar un saneamiento inicial aplicando las funciones de Pandas y NumPy.

Qué construir

Debes entregar un **informe de inspección inicial** basado en un Jupyter Notebook. Este documento debe mostrar el proceso de carga y el "diagnóstico" de tu dataset. El objetivo no es tener datos perfectos todavía, sino tener un mapa claro de qué datos tienes y qué problemas (nulos, tipos erróneos) deberás resolver.

Temas a cubrir en la entrega:

1. **Carga del Dataset:** Uso de `read_csv` (o similar) para traer tus datos al entorno.
2. **Estructura Base:** Reporte del número de filas y columnas (`shape`).
3. **Diagnóstico Estructural:** Uso de `info()` para identificar columnas con valores nulos y tipos de datos (`dtypes`) que no coincidan con la realidad (ej: números que Pandas lee como texto).
4. **Resumen Numérico:** Aplicación de `describe()` para entender los rangos de tus variables.
5. **Saneamiento Inicial:** Aplicación de al menos tres (3) filtros o selecciones booleanas que ayuden a "recortar" o limpiar el dataset (ej: eliminar filas con años fuera de rango, seleccionar solo una región de interés, o quitar columnas que claramente no aportan al problema de negocio).

Errores comunes a evitar

- **Ignorar los tipos de datos:** Es muy común intentar sumar una columna y obtener un error porque Pandas la interpretó como `object` (texto). Revisa siempre el `info()`.
- **No guardar los cambios:** Recuerda que filtrar un DataFrame como `df[df['edad'] > 18]` no modifica el original a menos que lo asignes a una nueva variable o a la misma (`df = df[df['edad'] > 18]`).

Conexión con el entregable final

Este dataset "saneado" será tu materia prima para el resto del curso. En el próximo módulo, usarás este mismo archivo para crear gráficos profesionales con Matplotlib y Seaborn. ¡Asegúrate de que la estructura sea sólida hoy para que tus visualizaciones mañana no mientan!

Entregable

Instrucciones de la pre-entrega

Para cumplir con esta pre-entrega, sigue estos pasos detallados. Recuerda que no evaluamos solo el código, sino tu capacidad de "leer" lo que los datos te están diciendo.

1. **Preparación:** Asegúrate de tener tu dataset (en formato CSV o Excel) en la misma carpeta donde crearás tu Notebook.
2. **Carga e Inspección:**
 - Carga el dataset usando `import pandas as pd`.
 - Usa `.head()` para una inspección visual.
 - Usa `.shape` e `.info()` para ver el volumen de datos y los tipos de columnas.
3. **Perfilado Inicial:**
 - Calcula el total de valores nulos por columna.
 - Calcula las estadísticas descriptivas con `.describe()`.
4. **Saneamiento y Selección:**
 - Utiliza **filtros booleanos** (ej. `df[(df['col1'] > x) & (df['col2'] == 'y')]`) para crear un subconjunto de datos que sea útil para tu pregunta de investigación.
 - Si hay columnas innecesarias para tu análisis, elimina al menos una usando los métodos de selección vistos.
5. **Reflexión:**
 - Agrega una celda de Markdown al final resumiendo: ¿Qué problemas encontraste en los datos? ¿Qué columnas serán tus variables clave?
6. **Exportación:**
 - En Jupyter: `File -> Download as -> PDF via HTML` (o imprime la página como PDF).
 - Nombre sugerido del archivo: `Apellido_Nombre_Checkpoint1.pdf`

Qué tenés que entregar: Un archivo PDF que contenga la exportación de tu Jupyter Notebook (o el notebook convertido a PDF) donde se vea claramente el código ejecutado, los resultados (tablas/resúmenes) y tus comentarios explicativos. Asegurate de que el PDF incluya el diagnóstico de nulos, dtypes, describe y los filtros realizados.